KEYBOARD-FIRST / TOOLCHAIN MAP

全键盘化工作流是怎么实现的?

不是靠一个神奇软件，而是把桌面、编辑器、浏览器、终端都换成同一套 keyboard-first 逻辑。

窗口管理 / MODAL EDITING / LINK HINTS / CLI + AI AGENT

先记住一句话：这是一套分层栈，不是单点技巧

第 1 层

桌面 / 窗口管理
先把焦点移动、工作区切换、布局调整变成键盘动作

第 2 层

编辑器 / IDE
用 Vim 逻辑统一移动、搜索、选择、修改

第 3 层

浏览器导航
用 link hints 和滚动键位把网页重新变成可跳转界面

第 4 层

终端 / CLI / AI 工作流
命令、prompt、agent 和审查链路继续 keyboard-first

真正的统一点是让 `hjkl`、modal、leader、link hints 在不同软件里重复出现。

PART 01

四层 keyboard-first 栈

从桌面到底层，再一路串到浏览器与 AI 工作流

第 1 层：桌面先统一，工作流才不会断流

为什么从窗口管理开始

- 切窗口、切工作区、移动焦点如果还靠鼠标，编辑效率会被频繁打断
- 平铺窗口管理器适合把方向键统一成 `h/j/k/l` 风格
- 底层统一之后，上层编辑和浏览才会真正顺起来

文档入口：窗口管理器总览 / Niri / Sway / Arch Linux Dotfiles

TOOLS

River / Sway / Niri / Hyprland

ACTION MAP

聚焦移动、布局调整、工作区切换都绑定到键盘

RESULT

鼠标不再是切换上下文的默认入口

第 2 层：编辑器用 Vim 逻辑统一高频文本动作

Neovim

最纯粹的 modal editing，把移动、编辑、对象操作都收回键盘。

VSCode + Neovim

保留现代 IDE 生态，同时把主操作逻辑统一到 VIM 键位。

Zed

更现代的界面，但依然强调键盘控制与快速跳转。

刚开始别追求 100% 不碰鼠标，先迁移 **移动、搜索、切文件、切窗口** 这四类动作。

第 3 层：浏览器补齐之后，键盘体验才会闭环

先记住这几个动作

`f` 点链接　`j` / `k` 滚动　`d` / `u` 翻页

`t` / `T` 标签页

- Qutebrowser 是原生 keyboard-first 浏览器
- 更容易落地的方案通常是 Chrome / Firefox + Surfingkeys
- link hints 让网页元素重新变成“可跳转目标”，而不是只能点选的区域

> *很多人最后卡在浏览器里；这层一补齐，工作节奏才不会每次打开网页就重置。*

第 4 层：终端优先，进一步变成 CLI / AI agent 工作流

TERMINAL-FIRST

很多操作直接变成命令，而不是在 GUI 面板里来回点选。

AI AGENT-FIRST

任务被写成 prompt、命令、检查和审查链路，更容易调用与验证。

WHY IT MATTERS

键盘优先不只是少用鼠标，而是让工作流更可脚本化、更可组合。

这一层对应的文档方向是 **OpenCode、iKanban、多 agent 并行工作流**，也就是把“操作界面”继续抽象成可执行流程。

PART 02

补充问题与最短上手路径

按键显示工具是什么，以及第一次阅读该怎么走

如果你问的是“视频里的按键浮窗是什么？”

答案

wshowkeys_rs

它负责把已经按下的键实时显示出来，适合录教程、直播、演示快捷键，或者调试键位绑定。

别和工作流本身混在一起

- 全键盘化 = 输入方式与工具栈设计
- 按键显示 = 把输入过程可视化给观众看
- 一个是 workflow，本质上层；一个是 overlay，属于展示层

第一次接触这套体系，最短阅读路线是这样

1. **窗口管理器总览**：先理解桌面层为什么是起点
2. **VSCode + Neovim 或 Zed**：建立编辑器层习惯
3. **Surfingkeys 指南**：把浏览器补齐
4. **OpenCode / 更好的 AI IDE**：延伸到终端和 agent
5. **wshowkeys_rs**：只有在需要展示快捷键时再补

优点在于可以按层迁移：**每补上一层，就少一点鼠标依赖，也多一点逻辑统一。**

最后收束：不是一个工具，而是一整套统一逻辑

1	2	3	4
窗口层统一焦点移动	编辑层统一文本操作	浏览层统一网页导航	终端层统一命令与 agent

真正的目标不是“炫技”，而是让 **切窗口、写代码、看网页、跑任务** 都遵循同一套心智模型。